

Numerical Methods for Determination of Lefschetz thimbles

(Summer Project Report)



Department of Physical Sciences
Indian Institute of Science Education and Research (IISER)
Mohali

August 10, 2020

Author - R. Bharathkumar

Supervisor - Dr. Anosh Joseph

Abstract

In this report, we outline some of the numerical methods of determining thimbles. The thimble cooling approach is introduced, and a subtle issue that plagues the algorithm is discussed. A solution to the problem is provided, and the effectiveness of the solution is illustrated.

Acknowledgements

I sincerely thank Dr Anosh Joseph for giving me an opportunity to work on this project, his continuing support, and for multiple fruitful and enlightening meetings. Although I might not have been at my best productivity, these discussions greatly helped me in keeping myself grounded during these difficult times due to the pandemic.

The summer internship was in collaboration with Roshan Kaundinya, and it was a pleasure to work with him. In our conversations, he often brought fresh perspectives, which helped me strengthen my understanding of Lefschetz thimbles.

R. Bharathkumar
IISER Mohali
September 22, 2020

Contents

1	Introduction	5
2	Picard-Lefschetz theory	6
2.1	Lefschetz thimbles	7
2.2	Lefschetz Thimbles in Zero Dimensions	8
3	Thimble Cooling Approach	8
3.1	Domains of Failure	9
4	A Proposal for a New Algorithm	10
5	An Illustration	11
6	Discussion and Closing Remarks	12
A	Code	15

1 Introduction

Many physically interesting and relevant systems are characterized by complex actions – Yang-Mills theory in the theta vacuum, Chern-Simons gauge theories, chiral gauge theories, and QCD with chemical potential. Recent studies have worked on multiple frameworks to deal with these actions, the two most prominent being complex Langevin dynamics and the Lefschetz thimble formalism. While the former is a numerical framework utilizing stochastic evolution, the latter exploits the topological properties of the target space of degrees of freedom.

It all starts with the integration measure for path integrals involving the system

$$D\phi \exp(aS[\phi]), \quad (1)$$

where we have schematically denoted all degrees of freedom of the system (fields) with ϕ . $S[\phi]$ is the action, and a is a constant that depends on the choice of units and the regime in which we choose to work. For Euclidean path integrals with $\hbar = 1$, $a = -1$. We choose to deal with field-theoretical systems in our discussions. However, the arguments can be easily translated into the language of systems involving other path integrals (e.g. quantum harmonic oscillators).

Consider a system with fields which, for ease of discussions, are taken to be real variables. The parameters of the system can in general be taken to be complex. Although this might seem ad hoc, there are many examples of such systems. The most straightforward examples involve the fermion determinant, which could end up taking complex values. Due to the presence of these complex terms, the integration measure in general ends up picking, putting us in the domain of oscillatory integrals.

Oscillatory integrals come with a plethora of rich mathematical structures, but the more pragmatic and physically interesting aspect of these integrals is that they are immensely difficult to compute numerically. The oscillatory nature of the measure leads to problems with the precision of numerical results for these integrals, and is called the sign problem. Although the sign problem appears in many different languages, the underlying problems remain the same.

Complex Langevin dynamics and Lefschetz thimbles formalism provide a solid framework to deal with these issues, but have their own problems. Particular for Lefschetz thimbles, although they are mathematically exact, finding these thimbles for specific actions remains a formidable task due to the complex nature of the differential equations that govern them. Yet, these thimbles can be found numerically by exploiting the principle properties of these thimbles, and there has been many successes in implementing these algorithms recently.

In this summer project, we looked at a specific numerical method for determining these thimbles. Called the thimble cooling approach, it utilises stochastic evolution

and the defining properties of thimbles to “cool” an initial set of points to a configuration where these points lie on the required integration contours [1]. Although a solid algorithm that provides strong results, it is plagued by a subtle problem, which prevents points too far away from the critical points of the action from cooling.

After providing a short tour of the theoretical foundations of Lefschetz thimbles mirroring the discussion in Ref. [2], we review the thimble cooling approach. A quick discussion of the issue with the algorithm is provided, followed by a new approach that replaces the initial evolution of the points with a honeycomb-like grid over which the points move in small steps to sufficiently minimise the distance from the thimble curves after which thimble cooling can be trusted and takes over. A sample code for the quartic model is provided in the appendix.

2 Picard-Lefschetz Theory

Picard-Lefschetz theory, a complex extension of Morse theory, deals with deducing topological properties of a complex manifold based on a special class of functions defined on these manifolds. These functions, referred to as Morse functions, are such that the critical points on the manifold are non-degenerate.

Take the action $S(\phi)$, where the space of ϕ is some real manifold $\mathcal{M}_{\mathbb{R}}$. We extend this into a function on $\mathcal{M}_{\mathbb{C}}$, which is the complexification of $\mathcal{M}_{\mathbb{R}}$. Assuming S is analytic in ϕ , we define two functions h and g , such that

$$h = -\text{Re}(S(\phi)) = -\frac{S + \bar{S}}{2}, \quad (2)$$

$$f = -\text{Im}(S(\phi)) = -\frac{S - \bar{S}}{2i}. \quad (3)$$

Let a point p be a non-degenerate critical point of h . By the requirement of analyticity, p is also a critical point of S . Consider the flow equations

$$\frac{dz^i}{dt} = -2g^{i\bar{j}} \frac{\partial h}{\partial \bar{z}^j} = -g^{i\bar{j}} \frac{\partial \bar{S}}{\partial \bar{z}^j}, \quad (4)$$

$$\frac{d\bar{z}^j}{dt} = -2g^{i\bar{j}} \frac{\partial h}{\partial z^i} = -g^{i\bar{j}} \frac{\partial S}{\partial z^i} \quad (5)$$

where $g^{i\bar{j}}$ is the Kahler metric on the manifold $\mathcal{M}_{\mathbb{C}}$.

Along this flow, the real part of the action increases monotonically, while the

imaginary part remains constant

$$\frac{dh}{dt} = -2g^{i\bar{j}}\partial_i h \bar{\partial}_{\bar{j}} h = -|\nabla h|^2 \leq 0 \Rightarrow \frac{d}{dt}(S + \bar{S}) \geq 0, \quad (6)$$

$$\frac{df}{dt} = \frac{dz^i}{dt}\partial_i f + \frac{d\bar{z}^{\bar{j}}}{dt}\bar{\partial}_{\bar{j}} f = -\frac{g^{i\bar{j}}}{2i}(\partial_i S \bar{\partial}_{\bar{j}} S + \partial_i S(-\bar{\partial}_{\bar{j}} S)) = 0. \quad (7)$$

These two properties are desirable since they (at least pre-emptively) solve the sign problem.

2.1 Lefschetz Thimbles

Solutions to the flow equation (4) can be used to define a relative homology of $\mathcal{M}_{\mathbb{C}}$. With this, we can define new integration cycles as elements of this relative homology which by construction satisfy: (i.) the integral in Eq. (1) converges exponentially, and (ii.) the integrand becomes non-oscillatory. Let p_i denote the non-degenerate critical points of the action. Then these homology elements, denoted as $\mathcal{J}_i$ are given by

$$\mathcal{J}_i = \left\{ \gamma(0) \in \mathcal{M}_{\mathbb{C}} \left| \frac{d\gamma}{dt} = \{f, \gamma\}_i, \quad \lim_{t \rightarrow -\infty} \gamma = p_i \right. \right\}, \quad (8)$$

where

$$\{f, \gamma\} = -2ig^{k\bar{l}}(\partial_k f \bar{\partial}_{\bar{l}} \gamma - \bar{\partial}_{\bar{l}} f \partial_k \gamma). \quad (9)$$

The original integration cycle $\mathcal{M}_{\mathbb{R}}$ needs to be written in terms of these $\mathcal{J}_i$ s. Regarding $\mathcal{M}_{\mathbb{R}}$ as an element of the n -th homology $H_n(\mathcal{M}_{\mathbb{C}}, (\mathcal{M}_{\mathbb{C}})_{-T}; \mathbb{Z})^1$ generated by the $\mathcal{J}_i$ s, we can write

$$\mathcal{M}_{\mathbb{R}} = \sum_i n_i \mathcal{J}_i, \quad n_i \in \mathbb{Z}. \quad (10)$$

With this, the partition function now becomes

$$Z = \sum_i n_i \int_{\mathcal{J}_i} e^{-S(\phi)} \mathcal{D}\phi. \quad (11)$$

The constants n_i are yet unknown. Define the anti-thimble associated with $\mathcal{J}_i$ as

$$\mathcal{K}_i = \left\{ \gamma(0) \in \mathcal{M}_{\mathbb{C}} \left| \frac{d\gamma}{dt} = \{f, \gamma\}_i, \quad \lim_{t \rightarrow +\infty} \gamma = p_i \right. \right\}. \quad (12)$$

Since by definition, $h(x) \leq h(p_i)$ for all $x \in \mathcal{J}_i$ and $h(x) \geq h(p_i)$ for all $x \in \mathcal{K}_i$, and the equality holds only at $z = p_i$, the thimble and the anti-thimble intersect

¹Here, $(\mathcal{M}_{\mathbb{C}})_{-T} = \{x \in \mathcal{M}_{\mathbb{C}} | h(x) \leq -T\}$.

only at p_i . Further, assuming that any two critical points are not connected by the *gradient flow*, $\mathcal{J}_i$ and $\mathcal{K}_j$ intersect only if $i = j$

$$\langle \mathcal{J}_i, \mathcal{K}_j \rangle = \delta_{ij}, \quad (13)$$

leading to

$$n_i = \langle \mathcal{M}_{\mathbb{R}}, \mathcal{K}_i \rangle. \quad (14)$$

This gives the weight of the integral on $\mathcal{J}_i$ in terms of the number of intersections between the original integration cycle and the anti-thimble.

2.2 Lefschetz Thimbles in Zero Dimensions

In zero dimensions, the thimble construction can be put in easier terms. Consider an action depending on the real field ϕ . The partition function is

$$Z = \int_{\mathbb{R}} e^{-S(\phi)} \mathcal{D}\phi. \quad (15)$$

The flow equation, Eq. (4), becomes

$$\frac{d\phi}{dt} = \overline{\left(\frac{\delta S}{\delta \phi} \right)}. \quad (16)$$

Under the boundary condition $\phi(-\infty) = p_i$, the solution to the above equation gives the thimble associated with the critical point p_i . The path integral is now

$$Z = \sum_i \langle \mathbb{R}, \mathcal{K}_i \rangle \int_{\mathcal{J}_i} e^{-S(\phi)} \mathcal{D}\phi \quad (17)$$

and $\mathcal{K}_i$ is given by the solution to Eq. (16) under the boundary condition $\phi(+\infty) = p_i$.

3 Thimble Cooling Approach

In Ref. [1], the authors proposed a new method to numerically find Lefschetz thimbles. Called the “thimble cooling” approach, it provides a useful and efficient technique to identify thimbles and anti-thimbles.

The starting point for the construction is the distance function M_σ . For a given point $p = x + iy$ and a critical point z_σ , the distance function evaluates the difference

between the imaginary part of the action at p and the imaginary part of the action at z_σ

$$M_\sigma(x, y) = |\text{Im}S(x + iy) - \text{Im}S(z_\sigma)|^2. \quad (18)$$

The goal of the thimble cooling approach is to iteratively minimize the above defined distance. To achieve this, the authors propose an evolution utilizing the gradient of the distance function

$$\dot{x} = -\frac{\partial M_\sigma(x, y)}{\partial x}, \quad (19)$$

$$\dot{y} = -\frac{\partial M_\sigma(x, y)}{\partial y}. \quad (20)$$

Thus, one starts with a point at a random location. Iteratively, the points are evolved using the above gradients, akin to the Langevin flow that utilizes the gradient of the action.

3.1 Domains of Failure

Although the thimble cooling method provides an excellent method for finding the thimbles, the method fails for actions that have a “large” distance function. As an example, let us consider a model with a quartic potential

$$S = \frac{\alpha}{2}z^2 + \frac{\beta}{4}z^4. \quad (21)$$

Schematically, the distance function goes as

$$M \sim x^8, y^8. \quad (22)$$

The gradient thus schematically depends as

$$\frac{\partial M_\sigma(x, y)}{\partial x} \sim x^7, y^8, \quad (23)$$

$$\frac{\partial M_\sigma(x, y)}{\partial y} \sim x^8, y^7. \quad (24)$$

As an illustration, take the critical point to be the one at the origin. For the initial point at $x = 5, y = 0$, the gradient of the distance function is $O(10^5)$. Even after accounting for cancellations from negative terms in the action, the gradient is too large. This results in the point in being flung away at an even further distance. Although in theory the point would eventually end up close to the thimble, it comes

at a huge cost of time. Plus, the gradient of the distance quickly grows and leads to overflow errors.

Thus, unless properly modulated, the evolution for points sufficiently away from the critical point can end up featuring large steps. This would result in a rather chaotic evolution where the points end up going farther from the thimbles and anti-thimbles, eventually leading to overflow errors.

4 A Proposal for a New Algorithm

In order to solve the issues that arise with a large distance function in the thimble cooling algorithm discussed in Sec. 3.1, we propose an alternative solution. The algorithm attempts to solve the issues in the initial stages of the thimble cooling approach by starting with an evolution that does not have a step size dependent on the distance function. Instead, it utilizes a fixed honeycomb-like grid on which the points move by a fixed distance. The algorithm is as follows:

1. Choose a starting point
2. Determine the N directions along which the point moves
3. Calculate the distance of the possible new locations of the point from the thimble/anti-thimble
4. Choose the location with the least distance to the thimble/anti-thimble
5. Repeat steps 1-4 till the distance $M < \delta$

In the above steps, the quantity δ is a threshold, and N is the number of directions along which the points move. The latter would be explained in more detail below. There is a natural choice for the threshold δ and the step size; for an equally spaced grid of points, it is the spacing between two adjacent points.

The algorithm, much like the thimble cooling approach, starts with an initial set of points. Every point has N possible directions that it can move along. As an example, we use $N = 3$. Thus, every point can move along three equally spaced directions (illustrated in Fig. 1). Each point is individually moved along the three directions, and the distance of the new locations from the thimble/anti-thimble is calculated. The configuration which is the closest to the thimble/anti-thimble is chosen, and the process is repeated. When the points are close enough to the thimble, so as to avoid large jumps resulting from the large value of the distance function, the algorithm

switches over to a faster algorithm of choice, such as the thimble cooling approach, and continues the evolution.

As an additional complication to the algorithm, one can choose “smudged” directions instead of a fixed set of directions (illustrated in Fig. 1). In this case, the N angles are chosen at random, such that on an average, they are equally spaced. Although this choice is not entirely necessary, it avoids the possibility of the points running in loops. This needs to be taken care only when $2N/(N-2)$ is an integer (e.g. $N = 3, 4, 6$).

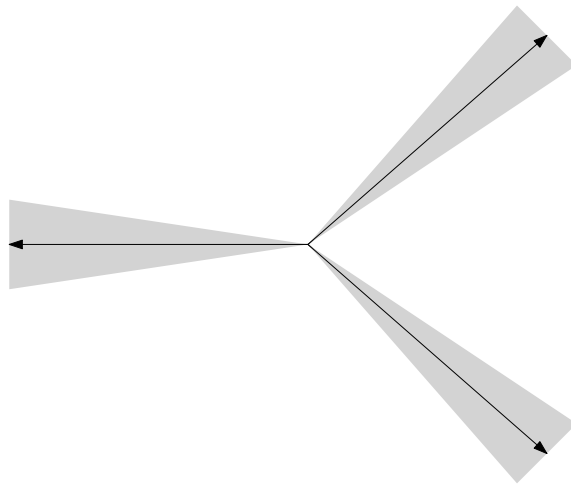


Figure 1: Illustration of the directions along which the point can move, for $N = 3$

5 An Illustration

In order to demonstrate the effectiveness of the algorithm, we provide the example of thimbles and anti-thimbles for the model

$$S = \frac{a + ib}{2} z^2 + \frac{c + id}{4} z^4. \quad (25)$$

We have chosen the critical point at the origin. The evolution was completely performed using the proposed algorithm, although in actual simulations, one might prefer to choose a faster algorithm once the points are closer to the curves. The results for $a = b = c = d = 1$ are shown in Fig. 2, contrasted with the expected result from

analytical computations in Fig. 3. A similar example is given for $a = b = c = 1, d = 0$ in Figs. 4 and 5

As is evident, the algorithm gives thimbles, anti-thimbles, and ghosts. Due to the use of the distance function, this is inevitable. However, the points can be filtered out post evolution by comparing to the real part of the action at the critical point.

6 Discussion and Closing Remarks

In this report, we have looked at numerical methods for finding Lefschetz thimbles. After giving a brief introduction to the theoretical underpinnings of the formalism, we introduced the Thimble Cooling approach. We discussed issues that arise while using the algorithm for points away from critical points, and provided a solution to the problem that supplements the initial steps of the algorithm. The solution brings the points close enough to the curves so as to reduce the possibility of random jumps, after which the cooling algorithm can be trusted and used.

The most obvious and immediate extension would be to models involving fermions. The algorithm makes no assumption about the nature of the action, and thus should be generalisable to fermionic systems, provided the logarithm that turns up is taken care of properly.

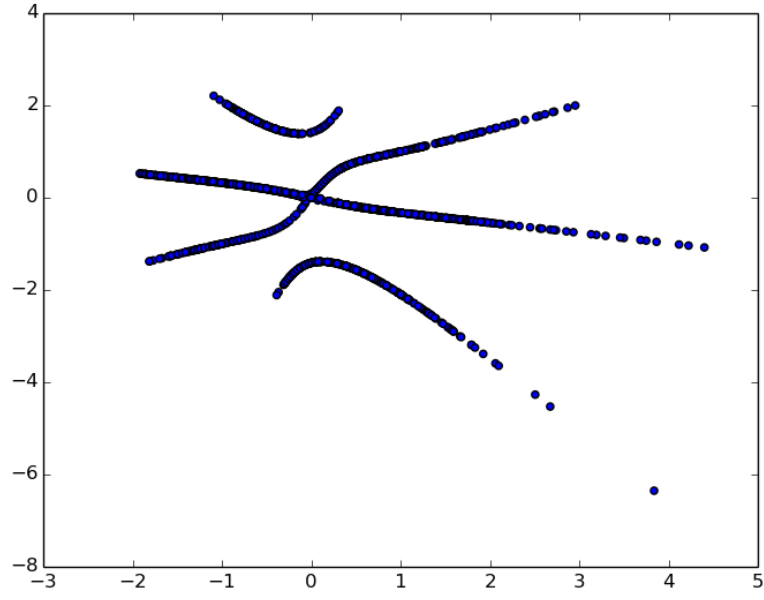


Figure 2: The results of evolution using the new algorithm, for the choice of parameters $a = 1, b = 1, c = 1, d = 1$.

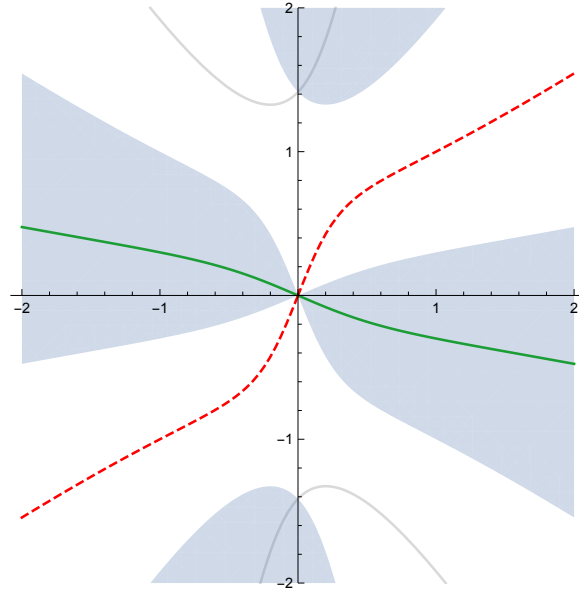


Figure 3: The results of evolution using the new algorithm, for the choice of parameters $a = 1, b = 1, c = 1, d = 1$.

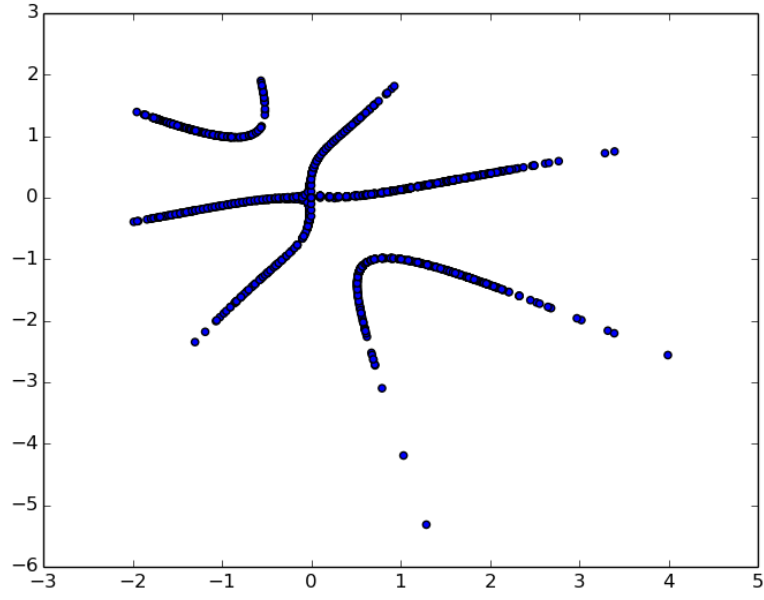


Figure 4: The results of evolution using the new algorithm, for the choice of parameters $a = 1, b = 1, c = 1, d = 0$.

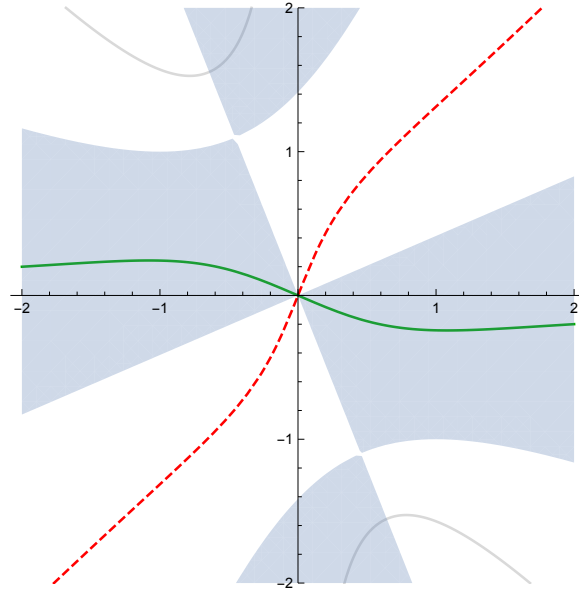


Figure 5: The results of evolution using the new algorithm, for the choice of parameters $a = 1, b = 1, c = 1, d = 0$.

A Code

```
import numpy as np
import matplotlib.pyplot as plt

#Setting the parameters of the model
a = 1.0
b = 1.0
c = 1.0
d = 1.0

# Setting up the action of the model
def RealAction(x,y):
    RealAction = b*(3*(x**3)*y - 3*x*(y**3))/4 + a*(x**4 + y**4 -
        6*(x**2)*(y**2))/4 + d*(2*x*y)/2 + c*(x**2 - y**2)/2
    return RealAction

def ImaginaryAction(x,y):
    ImaginaryAction = a*(3*(x**3)*y - 3*x*(y**3))/4 + b*(x**4 + y**4 -
        6*(x**2)*(y**2))/4 + c*(2*x*y)/2 + d*(x**2 - y**2)/2
    return ImaginaryAction

# Choosing the critical point of the action
RealCritical = 0.0
ImaginaryCritical = 0.0

# Distance function, defined as the absolute value of the difference
# between the
# imaginary part of the action at a random point, and imaginary part of
# the action
# at the critical point
def DistanceFunctionM(x,y):
    distance = (abs(ImaginaryAction(x,y) -
        ImaginaryAction(RealCritical,ImaginaryCritical)))*2
    return distance

# To determine if the point belongs on a thimble or an anti-thimble
# True means it is thimble. False means it is anti-thimble
def thimbleOrNotThimble(x,y):
```

```

    if RealAction(x,y)>RealAction(RealCritical,ImaginaryCritical):
        return True
    return False

def nextStep(x,y):
    dist = DistanceFunctionM(x,y)
    if dist>10000:
        print x,y,"discarded"
        return 0,0
    if dist>=resolution:
        tau = resolution
    else:
        tau = dist
    fudge1 = 2*0.1*np.random.random_sample() - 0.1
    x1 = x + tau*np.cos(np.pi/6 + fudge1)
    y1 = y + tau*np.sin(np.pi/6 + fudge1)
    fudge2 = 2*0.1*np.random.random_sample() - 0.1
    x2 = x + tau*np.cos(5*np.pi/3 + fudge2)
    y2 = y + tau*np.sin(5*np.pi/3 + fudge2)
    fudge3 = 2*0.1*np.random.random_sample() - 0.1
    x3 = x + tau*np.cos(3*np.pi/2 + fudge3)
    y3 = y + tau*np.sin(3*np.pi/2 + fudge3)
    #
    dist1 = DistanceFunctionM(x1,y1)
    dist2 = DistanceFunctionM(x2,y2)
    dist3 = DistanceFunctionM(x3,y3)
    #
    if dist1>dist2:
        if dist2>dist3:
            return x3,y3
        else:
            return x2,y2
    else:
        if dist1>dist3:
            return x3,y3
        else:
            return x1,y1

# Setting up the evolution, with numberOfPoints number of points
tolerance = 0.00001 #how close you want the point to lie near the curve
squareSize = 2 # size of the square of random points

```

```

resolution = 0.1

initialPoints = []

for i in range(0,int(2*squareSize/resolution)):
    for j in range(0,int(2*squareSize/resolution)):
        initialPoints.append((-squareSize+(i*resolution),-squareSize+(j*resolution)))

finalPoints = []

for point in initialPoints:
    x=point[0]
    y=point[1]
    while DistanceFunctionM(x,y)>tolerance:
        x,y = nextStep(x,y)
        finalPoints.append((x,y))

xValues = []
yValues = []

for point in finalPoints:
    xValues.append(point[0])
    yValues.append(point[1])

plt.scatter(xValues,yValues)
plt.show()

```

References

- [1] S. Bluecher, J. M. Pawłowski, M. Scherzer, M. Schlosser, I.-O. Stamatescu, S. Syrkowski, and F. P. Ziegler, “Reweighting Lefschetz Thimbles,” *SciPost Phys.* **5** no. 5, (2018) 044, [arXiv:1803.08418 \[hep-lat\]](#).
- [2] R. Bharathkumar, “The Sign Problem and Lefschetz Thimbles (Summer Project Report),”.